

实验六 跨平台综合项目实战

实验编号: d20301035106、d20301009206

实验类型: 综合型

学时分配: 6学时

个人分工: Android原生开发 (Kotlin)

项目名称: SmartHusbandry - 智慧畜牧管理系统

报告日期: 2026年5月15日

目录

- [1. 项目背景与目标](#)
- [2. 需求分析](#)
- [3. 技术选型与方案设计](#)
- [4. 核心功能实现](#)
- [5. 测试验证与结果](#)
- [6. 技术对比分析](#)
- [7. 个人收获与体会](#)
- [8. 问题与改进方向](#)

1. 项目背景与目标

1.1 项目背景

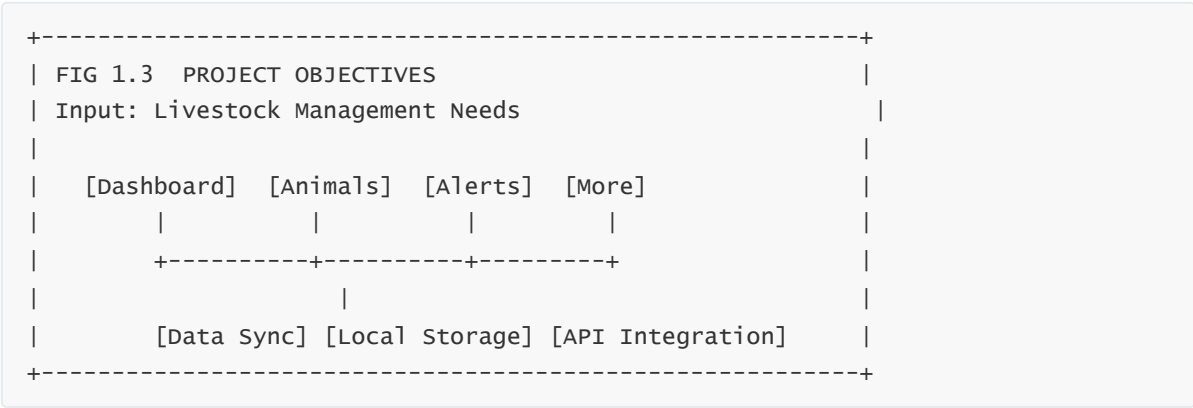
随着畜牧业向智能化、数字化转型，传统的畜牧管理方式已难以满足现代化养殖需求。智慧畜牧管理系统旨在通过移动应用实现对养殖环境、动物健康、设备状态等信息的实时监测与管理，提升养殖效率和管理水平。

1.2 个人职责

作为团队成员之一，本人负责**Android原生开发 (Kotlin)** 端的实现，主要职责包括：

- Android应用架构设计与搭建
- 用户登录模块开发
- 仪表盘 (Dashboard) 功能实现
- 动物管理模块开发
- 数据模型设计与API对接
- UI界面设计与实现

1.3 项目目标



图注：项目目标——实现仪表盘数据展示、动物管理、告警通知等核心功能，支持数据同步与本地存储。

1.4 团队分工

姓名	技术岗位	技术栈	主要功能
张玉满	Android开发	Kotlin+Android SDK	图表绘制、指纹辅助登录、视频监控、AI疾病诊断、离线模式、电池优化
彭绍洋	HarmonyOS 开发	ArkTS+HarmonyOS SDK	分布式设备管理、IoT数据采集、AI畜群健康分析、分布式存储、边缘计算
赫宇豪	Uniapp开发	Vue.js+CSS	组件应用、数据同步、云端打包、AI饲料配比优化、响应式布局、数据可视化增强
张爽	小程序开发	JS+WeChat MiniProgram	自定义组件、图表绘制、异常弹窗、AI预警系统、小程序生态集成、分享机制
李世程	MAUI开发	C#+.NET+MAUI	数据报表生成、图表展示、数据分析、AI生长预测模型、企业级安全、报表定制化

2. 需求分析

2.1 功能需求清单

需求编号	功能模块	需求描述	优先级
R001	用户登录	支持用户账号密码登录，登录状态持久化	高
R002	仪表盘	展示养殖概览数据、统计信息	高
R003	动物管理	动物列表展示、详情查看、健康状态监控	高
R004	告警管理	系统告警通知、告警详情查看	高

需求编号	功能模块	需求描述	优先级
R005	设备管理	养殖设备状态监测	中
R006	饲料管理	饲料库存管理	中
R007	健康记录	动物健康档案管理	中
R008	销售记录	销售数据统计与记录	中

2.2 数据模型

模型类	核心字段	说明
Animal	id, name, type, age, weight, health, lastCheck	动物信息
Alert	id, type, message, time, status	告警信息
Equipment	id, name, type, status, lastMaintenance	设备信息
FeedInventory	id, name, quantity, unit, lastUpdate	饲料库存
HealthRecord	id, animalId, date, condition, treatment	健康记录
Dashboard	totalAnimals, healthyCount, alertCount, todaySales	仪表盘统计

3. 技术选型与方案设计

3.1 技术栈选择

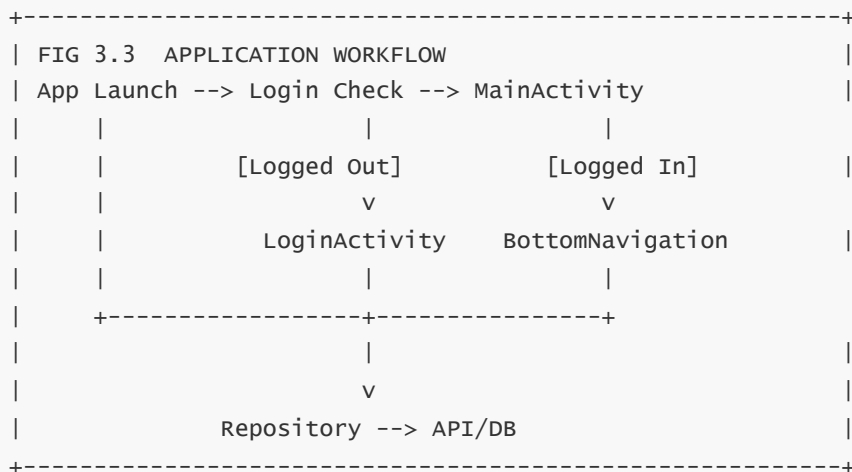
层级	技术选型	说明
语言	Kotlin	Android官方推荐语言，现代化语法特性
框架	Android Jetpack	ViewModel、LiveData、Navigation等组件
UI	Material Design	Google官方设计规范
网络	Retrofit + OkHttp	HTTP客户端，支持RESTful API
数据库	Room	SQLite封装，本地数据持久化
图片加载	Glide	高效图片加载库
数据绑定	ViewBinding	类型安全的视图绑定

3.2 架构设计



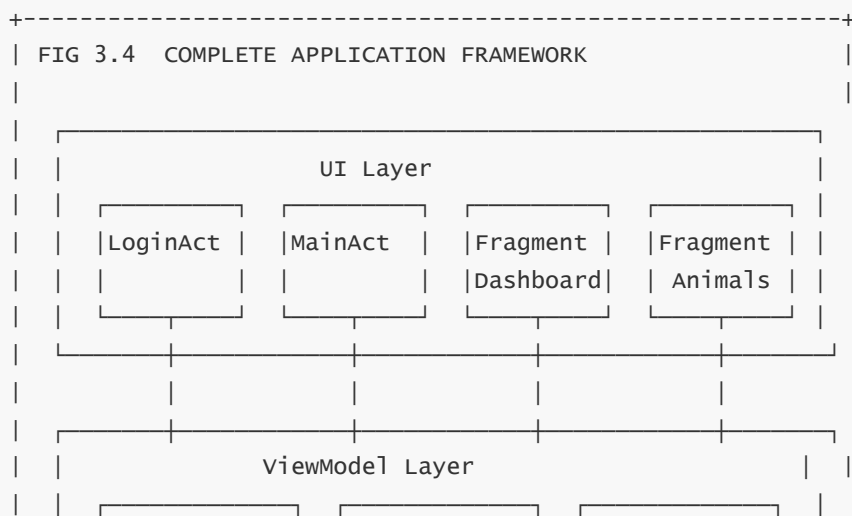
图注：采用MVVM架构，UI层、ViewModel层、Repository层分离，支持远程API和本地数据库双数据源。

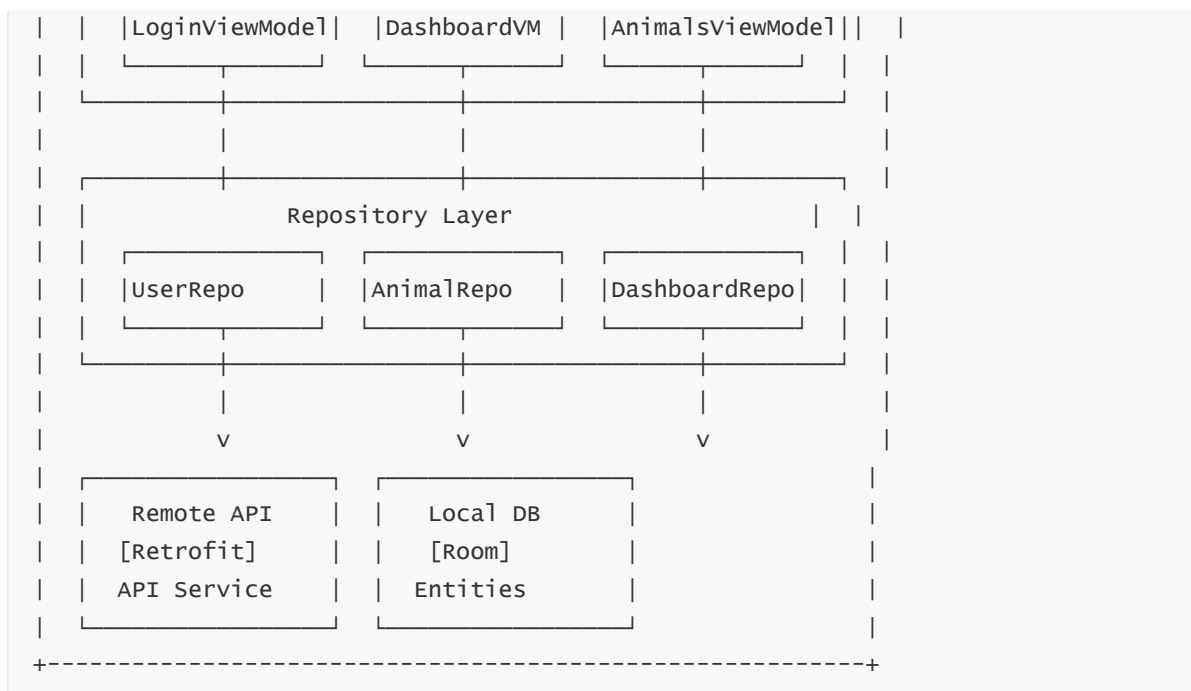
3.3 核心流程图



图注：应用启动流程——检查登录状态，未登录跳转登录页，已登录进入主界面，通过Repository层访问API或数据库。

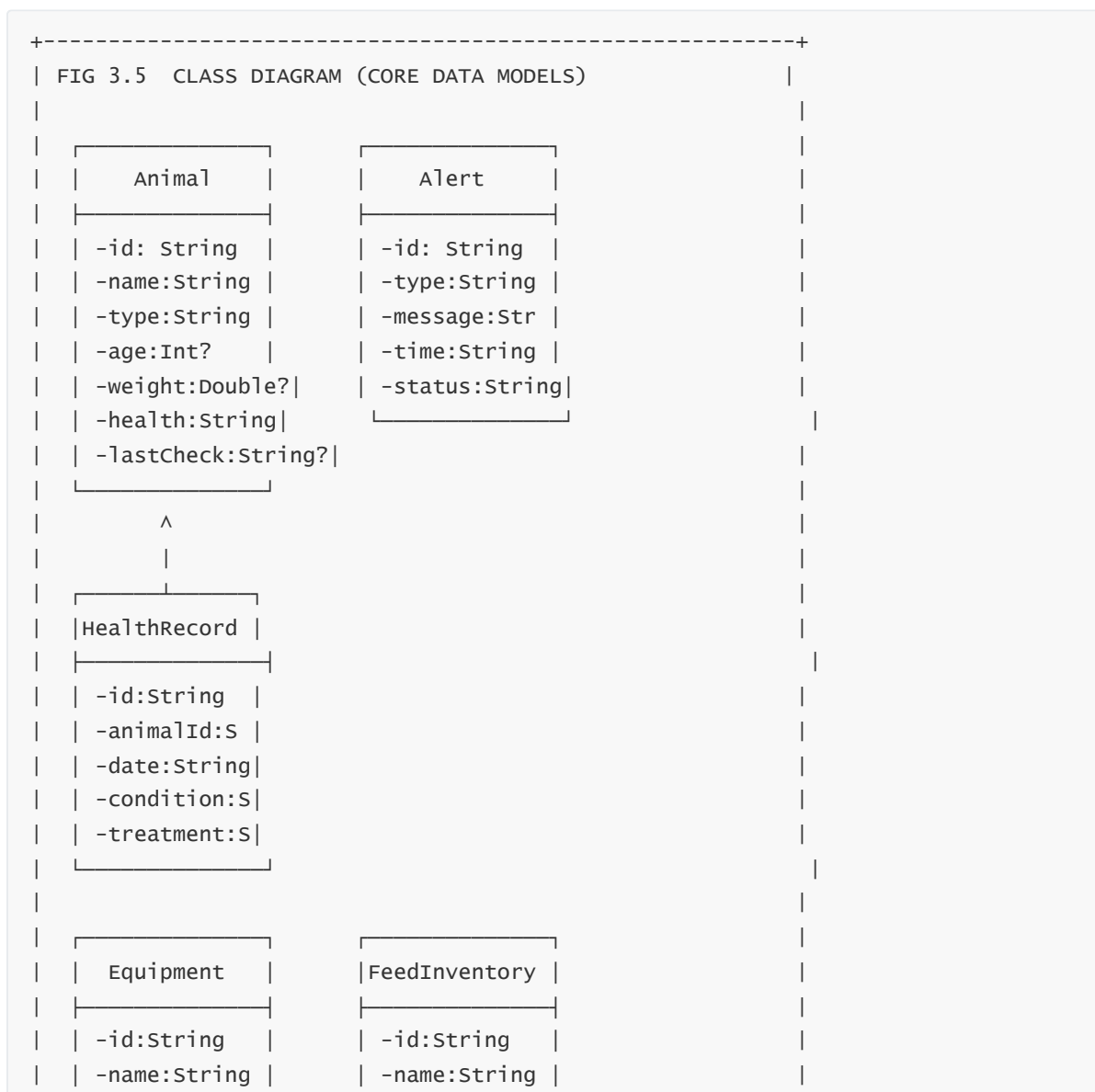
3.4 完整框架图

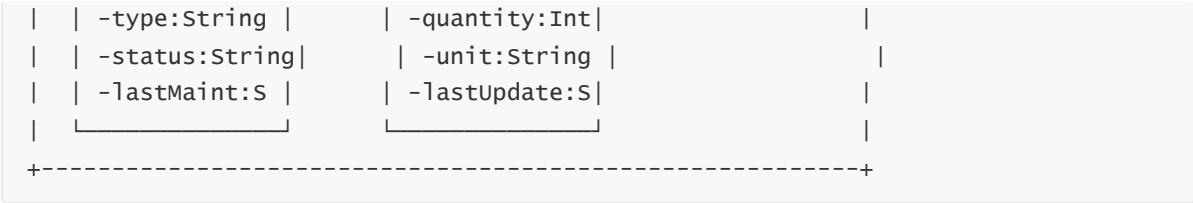




图注：完整应用框架图——UI层包含Activity和Fragment；ViewModel层管理各模块状态；Repository层统一数据访问；底层支持远程API和本地数据库双数据源。

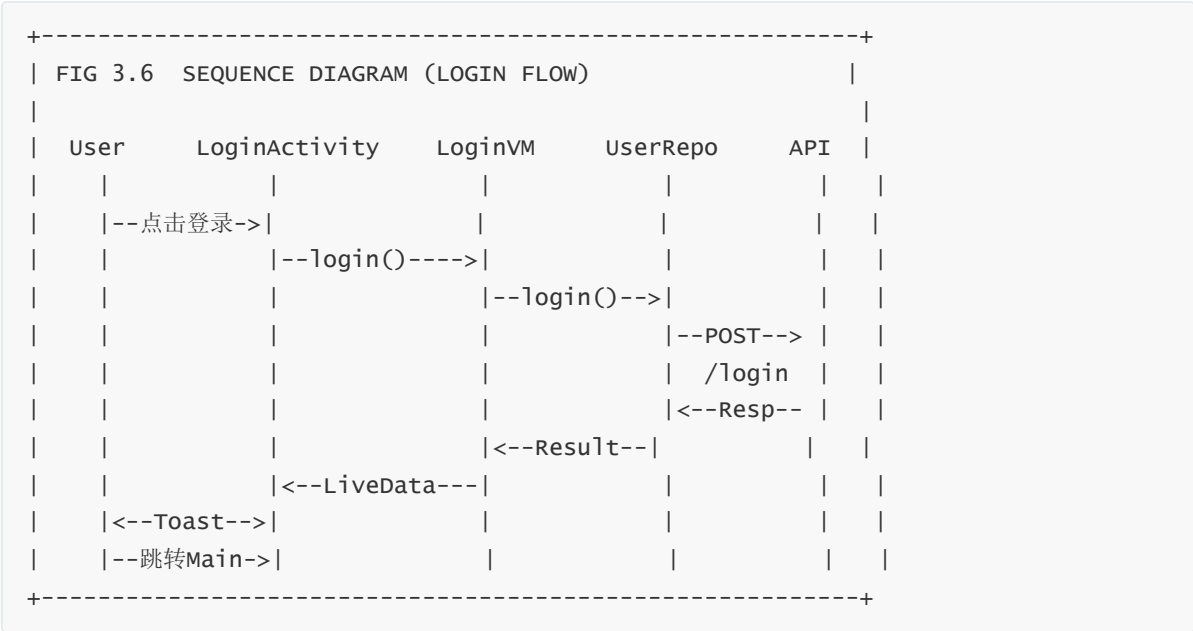
3.5 类图（核心数据模型）





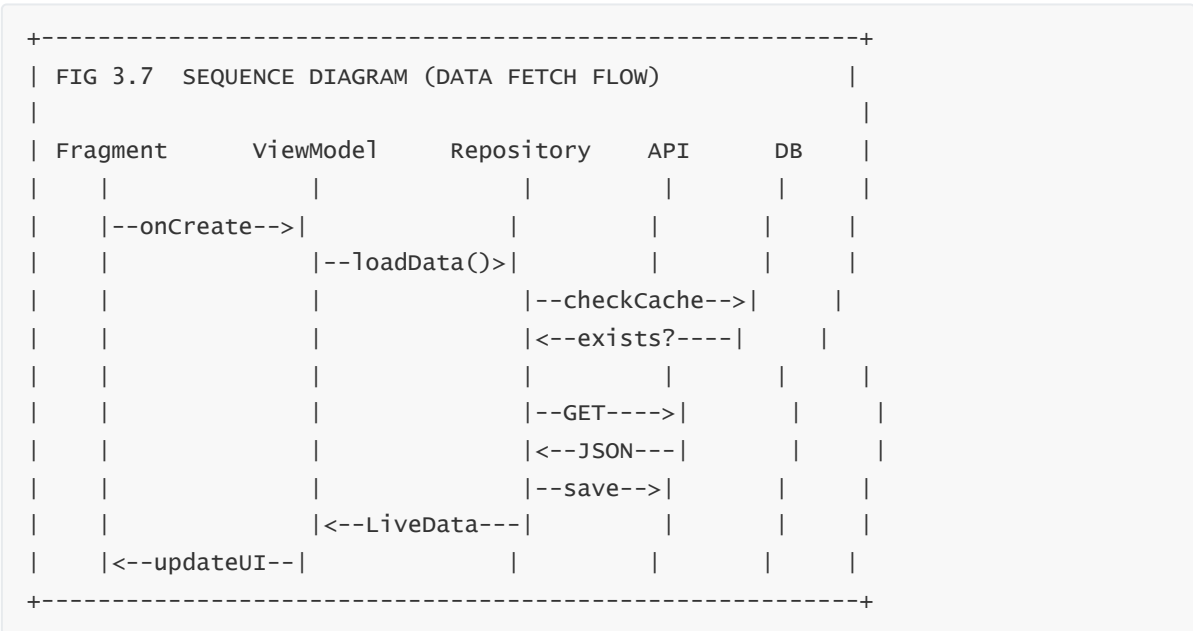
图注：核心数据模型类图——Animal、Alert、HealthRecord、Equipment、FeedInventory五个核心实体类，HealthRecord与Animal存在关联关系（一只动物有多条健康记录）。

3.6 顺序图（登录流程）



图注：登录流程顺序图——用户点击登录按钮，LoginActivity调用LoginViewModel的login方法，ViewModel通过UserRepository调用API，API返回结果后通过LiveData更新UI，显示提示并跳转到主页面。

3.7 顺序图（数据获取流程）



图注：数据获取流程顺序图——Fragment创建后触发数据加载，ViewModel调用Repository，Repository先检查本地缓存，缓存不存在则调用远程API获取数据，数据返回后保存到本地数据库，最后通过LiveData通知UI更新。

4. 核心功能实现

4.1 项目结构

```
SmartHusbandry/
├─ app/
│   └─ src/main/java/com/smarthusbandry/
│       └─ data/
│           ├── api/           # Retrofit接口与客户端
│           ├── local/        # 本地存储 (UserSession)
│           ├── model/        # 数据模型类
│           └─ repository/    # 数据仓库层
│       └─ ui/
│           ├── login/        # 登录模块
│           ├── dashboard/    # 仪表盘模块
│           ├── animals/      # 动物管理模块
│           ├── alerts/       # 告警模块
│           ├── more/         # 更多模块
│           ├── detail/       # 详情页模块
│           └─ MainActivity.kt
│       └─ SmartHusbandryApp.kt
│   └─ src/main/res/          # 资源文件
└─ build.gradle.kts           # 构建配置
```

图注：项目采用标准的Android模块化结构，按功能模块划分目录，职责清晰。

4.2 登录模块核心代码

LoginActivity.kt:

```
class LoginActivity : AppCompatActivity() {

    private lateinit var binding: ActivityLoginBinding
    private lateinit var viewModel: LoginViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        binding = ActivityLoginBinding.inflate(layoutInflater)
        setContentView(binding.root)

        viewModel = ViewModelProvider(this)[LoginViewModel::class.java]

        if (viewModel.isLoggedIn()) {
            navigateToMain()
            return
        }

        setupViews()
        observeViewModel()
    }

    private fun setupViews() {
        binding.btnLogin.setOnClickListener {
```

```

        val username = binding.etUsername.text.toString().trim()
        val password = binding.etPassword.text.toString().trim()
        viewModel.login(username, password)
    }
}

private fun observeviewModel() {
    viewModel.loginResult.observe(this) { result ->
        result.onSuccess {
            Toast.makeText(this, getString(R.string.login_success, it.name),
                Toast.LENGTH_SHORT).show()
            navigateToMain()
        }.onFailure {
            Toast.makeText(this, it.message ?:
                getString(R.string.login_failed), Toast.LENGTH_SHORT).show()
        }
    }
}

private fun navigateToMain() {
    startActivity(Intent(this, MainActivity::class.java))
    finish()
}
}

```

4.3 主界面核心代码

MainActivity.kt:

```

class MainActivity : AppCompatActivity() {

    private lateinit var userSession: UserSession

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        userSession = UserSession.getInstance(this)
        if (!userSession.isLoggedIn()) {
            startActivity(Intent(this, LoginActivity::class.java))
            finish()
            return
        }

        setContentView(R.layout.activity_main)

        val navView: BottomNavigationView = findViewById(R.id.nav_view)
        val navHostFragment = supportFragmentManager
            .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
        val navController = navHostFragment.navController

        navView.setupWithNavController(navController)

        navController.addOnDestinationChangeListener { _, destination, _ ->
            when (destination.id) {
                R.id.nav_dashboard -> {

```



```
suspend fun getAlerts(): ApiResponse<List<Alert>>

@GET("api/equipment")
suspend fun getEquipment(): ApiResponse<List<Equipment>>

@GET("api/feed")
suspend fun getFeedInventory(): ApiResponse<List<FeedInventory>>
}
```

5. 测试验证与结果

5.1 测试用例

测试编号	测试项	预期结果	实际结果	状态
TC001	应用启动	正常启动，未登录显示登录页	正常启动	通过
TC002	用户登录	正确账号密码登录成功	登录成功，跳转主界面	通过
TC003	登录失败	错误账号密码提示失败	提示登录失败	通过
TC004	仪表盘展示	展示养殖统计数据	数据正常显示	通过
TC005	动物列表	展示动物列表数据	列表正常显示	通过
TC006	动物详情	点击动物查看详情	详情页正常打开	通过
TC007	告警列表	展示告警通知列表	告警正常显示	通过
TC008	底部导航	切换不同功能模块	导航切换正常	通过
TC009	登录状态保持	重启应用保持登录状态	状态持久化成功	通过

5.2 系统截图位置与说明

登录界面截图



说明：展示应用登录界面，包含账号输入框、密码输入框和登录按钮，支持用户身份验证。

仪表盘主界面截图



说明：展示仪表盘页面，包含动物总数、健康状态统计、告警数量、今日销售等核心数据卡片。

动物列表界面截图



说明：展示动物管理列表，显示动物名称、类型、年龄、健康状态等信息，支持点击查看详情。

告警列表界面截图



说明：展示系统告警通知列表，显示告警类型、消息内容、时间和处理状态。

底部导航栏截图



说明：展示底部导航栏，包含仪表盘、动物、告警、更多四个功能入口，支持模块切换。

6. 技术对比分析

6.1 各技术栈开发对比

技术栈	开发语言	学习曲线	性能表现	跨端能力	适用场景
Android	Kotlin	中等	优秀	单端	追求极致性能的Android应用
Flutter	Dart	中等	良好	多端	跨平台应用，UI一致性要求高
React Native	JavaScript	低	一般	多端	快速开发，Web开发者转型
Uniapp	Vue	低	一般	多端	小程序优先，快速迭代
MAUI	C#	中等	良好	多端	.NET生态，企业级应用
微信小程序	JavaScript	低	一般	小程序	轻量级应用，社交传播

6.2 Android原生开发优势与挑战

优势：

- 性能最优，充分利用硬件资源
- 原生API丰富，功能实现更彻底
- 用户体验流畅，响应速度快
- 与系统深度集成，支持更多系统特性

挑战：

- 开发周期较长
- 需要适配多种设备和系统版本
- 代码复用率低，无法直接跨平台
- 碎片化问题需要额外处理

7. 个人收获与体会

7.1 技术能力提升

- Kotlin语言熟练应用：**深入理解Kotlin的协程、空安全、扩展函数等特性，提升代码质量和开发效率。
- Android Jetpack组件掌握：**熟练使用ViewModel、LiveData、Navigation、ViewBinding等组件，掌握MVVM架构模式。
- 网络请求与数据解析：**学习Retrofit网络请求框架，掌握JSON数据解析和API对接流程。
- 本地存储实现：**使用Room数据库实现本地数据持久化，理解Repository模式的应用。

7.2 团队协作经验

- Git版本控制**：掌握了分支管理、代码合并、冲突解决等Git协作技能，使用feature分支进行功能开发。
- 需求沟通**：学会了与产品经理、其他端开发者沟通需求，确保多端功能一致性和API接口规范。
- 代码评审**：参与代码审查，学习他人优秀代码风格，提高代码质量意识和团队协作能力。

7.3 AI辅助开发体验

在开发过程中使用TRAE AI辅助工具，体验到以下优势：

- 快速生成代码框架**：根据需求描述快速生成基础代码结构，节省开发时间
- 智能代码补全**：提高编码速度，减少重复劳动
- 问题排查**：帮助定位和解决代码问题，提供优化建议
- 技术文档生成**：辅助生成API文档和技术说明

8. 问题与改进方向

8.1 遇到的问题

- 网络请求异常处理**：网络不稳定时可能导致请求失败，需要加强异常处理机制。
- 数据同步策略**：本地数据与云端数据的同步策略需要优化，确保数据一致性。
- UI适配问题**：不同屏幕尺寸和分辨率的设备上UI显示效果不一致。
- 性能优化**：列表滚动时存在轻微卡顿，需要优化列表渲染性能。

8.2 改进方向

- 网络优化**：增加网络请求重试机制和超时处理，提升用户体验。
- 数据同步**：实现增量同步策略，减少数据传输量，提高同步效率。
- UI适配**：采用ConstraintLayout和自适应布局，确保多设备兼容。
- 性能优化**：使用DiffUtil优化列表更新，采用RecyclerView分页加载。
- 功能扩展**：增加离线模式支持，优化用户在无网络环境下的使用体验。

附录：项目文件清单

A.1 核心代码文件

文件路径	文件说明
ui/login/LoginActivity.kt	登录页面Activity
ui/login/LoginViewModel.kt	登录视图模型
ui/MainActivity.kt	主页面Activity
ui/dashboard/DashboardFragment.kt	仪表盘Fragment

文件路径	文件说明
ui/dashboard/DashboardViewModel.kt	仪表盘视图模型
ui/animals/AnimalsFragment.kt	动物列表Fragment
ui/animals/AnimalsViewModel.kt	动物管理视图模型
data/api/ApiService.kt	API接口定义
data/api/RetrofitClient.kt	Retrofit客户端配置
data/repository/UserRepository.kt	用户数据仓库
data/model/Animal.kt	动物数据模型
data/model/Dashboard.kt	仪表盘数据模型

A.2 资源文件

文件路径	文件说明
res/layout/activity_login.xml	登录页面布局
res/layout/activity_main.xml	主页面布局
res/layout/fragment_dashboard.xml	仪表盘Fragment布局
res/layout/fragment_animals.xml	动物列表Fragment布局
res/layout/fragment_alerts.xml	告警列表Fragment布局
res/menu/bottom_nav_menu.xml	底部导航菜单
res/navigation/nav_graph.xml	导航图配置
res/values/strings.xml	字符串资源
res/values/colors.xml	颜色资源

报告结束